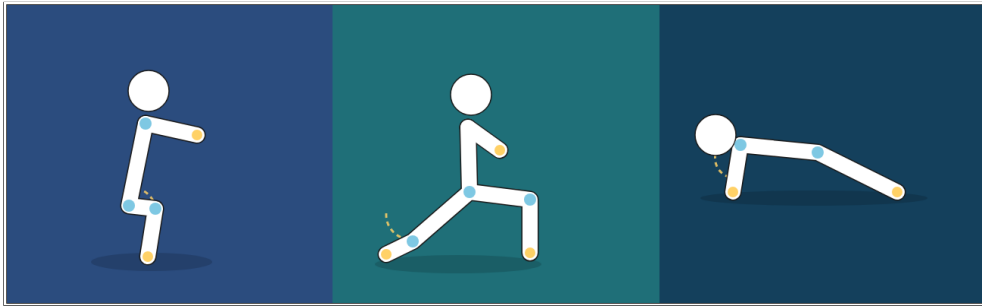




INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)
DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

LICENCIATURA EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA (LEIM)
UNIDADE CURRICULAR DE PROJETO (PRJ)

FitnessTracking: Aplicação Móvel para a Monitorização de Exercícios em Tempo Real



Rodrigo Correia (45155)

David Delgado (51598)

Orientador(es)

Professor Doutor Diogo Cabral

Julho, 2026

Resumo

Este projeto descreve o desenvolvimento de uma aplicação móvel nativa em Android para monitorização e correção da postura em tempo real durante exercícios físicos (como agachamentos, flexões de braços e afundos). A solução utiliza a biblioteca MediaPipe Pose Landmarker para detetar 33 pontos-chave do esqueleto corporal em tempo real, calculando de forma assíncrona os ângulos das articulações biomecânicas associadas.

Um motor baseado numa máquina de estados sequencial rastreia as fases concêntrica e excêntrica do movimento, gerindo a contagem e avaliando o ritmo e a amplitude de movimento (ROM) com base em limiares definidos. A aplicação fornece feedback sonoro imediato através do motor nativo Text-To-Speech (TTS), ajudando o atleta a corrigir desvios posturais sem distrações visuais.

A persistência dos treinos é sincronizada com uma base de dados na nuvem (Firebase Firestore), alimentando um painel de estatísticas interativo com gráficos customizados em Jetpack Compose Canvas que ilustram o progresso e a regularidade do utilizador. Testes experimentais confirmam o desempenho do processamento assíncrono local no dispositivo (edge computing) e a viabilidade das correções posturais imediatas, promovendo uma prática mais autónoma.

Palavras-chave: Estimativa de Pose, MediaPipe, Android, Text-To-Speech, Biomecânica, Firebase Firestore, Jetpack Compose.

Abstract

This project presents a native Android mobile application for real-time posture monitoring and correction during physical exercises (such as squats, push-ups, and lunges). The solution uses the MediaPipe Pose Landmarker library to detect 33 body skeleton keypoints in real time, asynchronously computing the associated biomechanical joint angles.

A sequential state-machine-based engine tracks the concentric and eccentric phases of the movement, managing repetition counting and evaluating cadence and range of motion (ROM) against defined thresholds. The app provides immediate auditory feedback through the native Text-To-Speech (TTS) engine, helping the athlete correct postural deviations without visual distractions.

Workout persistence is synchronized with a cloud database (Firebase Firestore), powering an interactive analytics dashboard with custom Jetpack Compose Canvas charts that illustrate the user's progress and consistency. Experimental tests confirm the performance of asynchronous on-device local processing (edge computing) and the feasibility of immediate postural corrections, promoting more autonomous practice.

Keywords: Pose Estimation, MediaPipe, Android, Text-To-Speech, Biomechanics, Firebase Firestore, Jetpack Compose.

Agradecimentos

Ao Professor Doutor Diogo Cabral, orientador deste projeto, pela disponibilidade e acompanhamento ao longo de todo o semestre.

Ao Professor Doutor António Teófilo, coordenador da Licenciatura em Engenharia Informática e Multimédia, pelo apoio institucional prestado ao longo do curso.

Ao Professor Doutor José Nascimento, Presidente do Instituto Superior de Engenharia de Lisboa, pelas condições proporcionadas para a realização deste trabalho.

Aos participantes dos testes de usabilidade, pelo tempo disponibilizado e pela paciência com as versões iniciais da aplicação.

Aos nossos familiares e amigos, que de forma direta ou indireta nos apoiaram ao longo de todos estes anos de estudo dedicados à licenciatura. A sua compreensão, incentivo constante e suporte incondicional foram o pilar fundamental para alcançarmos o sucesso neste projeto e concluirmos este ciclo.

Aos nossos colegas de curso, pelas valiosas trocas de ideias, partilha de conhecimento e por todos os momentos de companheirismo ao longo da jornada académica.

Índice

Resumo	i
Abstract	iii
Agradecimentos	v
Lista de Tabelas	ix
Lista de Figuras	xi
1 Introdução	1
2 Trabalho Relacionado	3
3 Modelo Proposto	5
3.1 Requisitos	5
3.2 Fundamentos Biomecânicos	6
3.2.1 Calibração e Validação dos Limiares	7
3.3 Abordagem de Avaliação de Forma	8
3.3.1 Exemplo Prático de uma Repetição	10
4 Implementação do Modelo	11
4.1 Arquitetura e Fluxo de Visão por Computador	11
4.2 Módulos de Código Principais	12
4.3 Integração de Base de Dados e Gamificação	15
5 Validação e Testes	17
5.1 Validação e Compilação Automatizada	17
5.2 Testes no Dispositivo e Limitações Físicas	17
5.3 Validação de Analítica e Estado Social	18
5.4 Testes de Usabilidade e Validação Prática	18
5.5 Produto Final	20
6 Conclusões e Trabalho Futuro	23
A Diagrama de Navegação da Interface	25

Bibliografia**27**

Lista de Tabelas

- 3.1 Exemplo prático de duas repetições de Agachamento e a pontuação resultante 10

Lista de Figuras

3.1	Topologia dos 33 landmarks do MediaPipe Pose [Google, 2024].	6
3.2	Máquina de estados da contagem de repetições.	9
4.1	Diagrama da arquitetura do projeto	11
4.2	Diagrama de classes UML do motor de avaliação de exercícios	14
4.3	Diagrama de classes UML da orquestração do treino.	14
4.4	Diagrama da estrutura e esquema de dados.	15
5.1	Distribuição dos resultados SUS por literacia tecnológica.	19
5.2	Ecrã de login e Menu inicial de seleção de treinos.	20
5.3	Ciclo de utilização da aplicação: Ecrã inicial, Execução do exercício (com HUD), Intervalo de descanso (Rest), Retoma do exercício e Painel de Resultados finais.	21
5.4	Ecrã de visualização de Resultados e Ranking de pontos.	21
A.1	Fluxo de navegação entre os principais ecrãs da aplicação.	26

Capítulo 1

Introdução

A prática de exercício físico tem vindo a registar um crescimento acentuado fora do ambiente tradicional dos ginásios, motivada pela conveniência dos treinos em casa. Contudo, a ausência de um profissional especializado (como um *personal trainer* ou fisioterapeuta) aumenta significativamente o risco de lesões musculoesqueléticas devido a posturas incorretas ou cadências desadequadas. A monitorização manual por parte do utilizador é difícil e distrai da execução do movimento.

O principal objetivo deste projeto consiste no desenvolvimento de uma aplicação móvel nativa em Android capaz de rastrear poses corporais, detetar e contar repetições de forma autónoma e fornecer feedback biomecânico corretivo em tempo real, utilizando apenas a câmara frontal do dispositivo móvel.

Os principais contributos deste trabalho são:

- Um motor de avaliação postural em tempo real baseado em regras cinemáticas e análise vetorial de ângulos articulares;
- Uma máquina de estados sequencial para a contagem de repetições e identificação de fases concêntrica/excêntrica;
- Um assistente de treino por voz nativo (Text-To-Speech) que providencia avisos de correção postural imediatos (com proteção de latência e sobreposição de fala);
- Um painel analítico interativo em Jetpack Compose Canvas para representação gráfica de regularidade sem dependências externas;
- Uma camada de persistência na nuvem e uma rede de amigos, com pedidos de amizade bidirecionais e deteção de estado online.

O presente relatório encontra-se estruturado da seguinte forma: o Capítulo 2 aborda as tecnologias de estimativa de pose e soluções similares. O Capítulo 3 apresenta os requisitos, fundamentos de ângulos articulares e modelação matemática do sistema. O Capítulo 4 detalha as opções tecnológicas e de código (Android, MediaPipe, Firebase). O Capítulo 5 reporta as experiências experimentais efetuadas e, por fim, o Capítulo 6 sumariza os resultados e perspetivas futuras.

Capítulo 2

Trabalho Relacionado

O rastreo de atividade humana por visão computacional tem sofrido grandes avanços científicos. As abordagens tradicionais focavam-se em sensores acoplados ao corpo (IMUs), os quais, apesar de precisos, revelavam-se desconfortáveis e dispendiosos para o utilizador comum. A transição para câmaras convencionais permitiu a emergência de frameworks como:

- **OpenPose [Cao et al., 2018]:** Baseado em redes convolucionais que detetam poses em múltiplas pessoas simultaneamente. Apesar do rigor, o seu elevado peso computacional exige processamento gráfico dedicado (GPUs), inviabilizando a execução local em dispositivos móveis comuns;
- **YOLO-pose [Maji et al., 2022]:** Focado na deteção rápida de caixas delimitadoras e poses combinadas. A sua precisão em orientações laterais de corpo inteiro pode, no entanto, oscilar, e exige inicializações complexas;
- **MediaPipe BlazePose [Bazarevsky et al., 2020]:** Otimizado especificamente para dispositivos móveis (Edge Computing). Utiliza uma arquitetura leve de duas fases (deteção de corpo na primeira frame e rastreo subsequente de 33 landmarks corporais em 3D), permitindo mais de 30 frames por segundo num Google Pixel 2.

Das três abordagens apresentadas, este projeto adota o MediaPipe BlazePose como motor de estimativa de pose, por ser a única otimizada para inferência local em dispositivos móveis sem necessidade de GPU dedicada. Ao contrário de sistemas que enviam as frames de vídeo para processamento num servidor central (gerando problemas de largura de banda e de privacidade do utilizador), este projeto adota processamento local no telemóvel (Edge Computing): as imagens da câmara nunca saem do dispositivo, e o processamento não depende de ligação à internet.

Ao nível da aplicação, o MediaPipe já foi utilizado para outros fins de análise postural: Garg, Saxena e Gupta [Garg et al., 2023] usam o MediaPipe para extrair os landmarks corporais e, a partir destes, classificar posturas de ioga estáticas em categorias através de uma rede neuronal convolucional.

Este projeto, no entanto, aplica o MediaPipe a um problema distinto: em vez de classificar uma pose estática isolada, avalia exercícios dinâmicos e repetitivos através de uma máquina de estados que acompanha a evolução do ângulo articular ao longo do tempo, contando repetições e penalizando desvios de amplitude e cadência.

Para além da escolha tecnológica, a avaliação da qualidade de cada repetição apoia-se também num pressuposto teórico: a biomecânica articular documentada na literatura de goniometria, nomeadamente a obra de Norkin e White [Norkin e White, 2016], detalhada na Secção 3.2 do Capítulo 3.

Capítulo 3

Modelo Proposto

Neste capítulo, formalizamos a estrutura funcional da aplicação, detalhando os requisitos do sistema, os limiares biomecânicos e a formulação da avaliação dos exercícios implementados.

3.1 Requisitos

O sistema foi modelado para dar suporte a atletas autónomos, identificando-se os seguintes requisitos:

- **Requisitos Funcionais:**

1. Captar poses corporais da câmara frontal em tempo real;
2. Detetar início da atividade quando o utilizador se encontra parado na pose inicial (calibração);
3. Contar repetições e fases do exercício (UP/DOWN/REST), ou o tempo sob tensão nos exercícios isométricos;
4. Gerar feedback áudio e visual imediato, com fins corretivos e motivacionais;
5. Sincronizar dados de treino do atleta com uma base de dados remota;
6. Apresentar um histórico interativo e gráficos semanais de progresso;
7. Gerir rede de amigos com envio e aceitação de pedidos de amizade.

- **Requisitos Não Funcionais:**

1. Tempo de inferência e cálculo de ângulos inferior a 30ms no dispositivo móvel;
2. Prevenção de bloqueio automático do ecrã (*screen wake lock*) durante treinos;
3. Interface visual adaptável a ecrãs de tablets (limitação de largura e centramento);
4. Segurança na autenticação de utilizadores.

3.2 Fundamentos Biomecânicos

Para validar a execução de cada exercício e garantir o rigor científico do sistema de avaliação, apoiamos-nos na obra de referência de Norkin e White [Norkin e White, 2016], que documenta a amplitude de movimento articular e a amplitude necessária para atividades funcionais concretas. Cada exercício de repetição (Agachamento, Flexão de Braços e Lunge) é avaliado por três limiares distintos: um limiar de extensão (UP), que marca a posição de topo do movimento e completa a repetição; um limiar ideal, que reflete a execução tecnicamente correta; e um limiar de contagem, mais permissivo, que define a partir de que ângulo a repetição é sequer registada (a lógica de deteção que usa estes três valores é detalhada na Secção 3.3). A Prancha Isométrica segue uma lógica diferente, sem esta distinção de limiares, também detalhada na Secção 3.3.

Os ângulos são sempre calculados a partir de três landmarks corporais fornecidos pelo modelo MediaPipe Pose [Bazarevsky et al., 2020], identificados por um índice fixo de 0 a 32, ilustrado na Figura 3.1. Por convenção, os índices pares correspondem ao lado direito do utilizador e os ímpares ao lado esquerdo; como o utilizador está de frente para a câmara, o seu lado direito aparece do lado esquerdo da imagem.

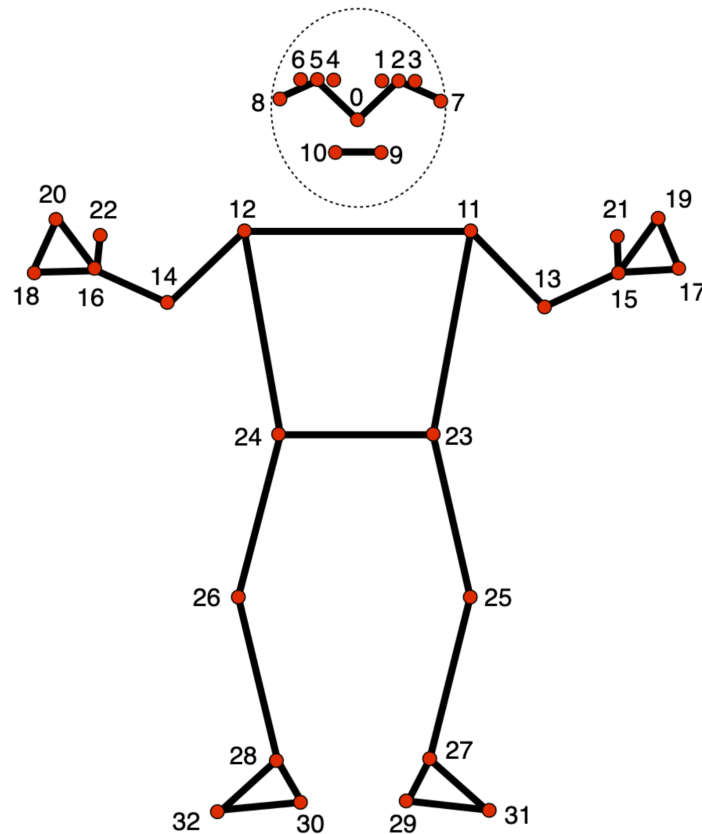


Figura 3.1: Topologia dos 33 landmarks do MediaPipe Pose [Google, 2024].

Definem-se os seguintes valores por exercício:

1. **Agachamento (Squat):** Analisa o ângulo θ formado pelo quadril (ponto 24), joelho (ponto 26, vértice) e tornozelo (ponto 28).
 - Limiar UP (Extensão): $\theta \geq 160^\circ$;
 - Limiar de Contagem: $\theta \leq 90^\circ$;
 - Limiar Ideal (Flexão): $\theta \leq 70^\circ$.
2. **Flexão de Braços (Push-Up):** Monitoriza o ângulo θ entre ombro (ponto 12), cotovelo (ponto 14, vértice) e pulso (ponto 16).
 - Limiar UP (Extensão): $\theta \geq 150^\circ$;
 - Limiar de Contagem: $\theta \leq 90^\circ$;
 - Limiar Ideal (Flexão): $\theta \leq 70^\circ$.
3. **Afundo (Lunge):** Monitoriza a flexão do joelho da perna da frente (e não da perna traseira). Ao contrário do Agachamento e da Flexão de Braços, este exercício é executado de frente para a câmara; a perna da frente, mais próxima da lente, tende a ocultar visualmente a perna de trás, pelo que apenas a perna da frente é medida.
 - Limiar UP (Extensão): $\theta \geq 160^\circ$;
 - Limiar de Contagem: $\theta \leq 115^\circ$;
 - Limiar Ideal (Flexão): $\theta \leq 90^\circ$.
4. **Prancha Isométrica (Plank):** Avalia a estabilização estática do core. O alinhamento dos landmarks do Ombro (12), Anca (24) e Tornozelo (28) deve manter-se dentro de uma janela de deteção entre 155° e 185° , com valor ideal em $\theta \approx 170^\circ$, ao longo do tempo total sob tensão.

3.2.1 Calibração e Validação dos Limiares

Os valores angulares apresentados não foram pensados todos da mesma forma. Numa primeira iteração, foram fixados de forma empírica: experimentámos em frente à câmara, ajustando os graus até a contagem de repetições parecer fiável e a exigência de profundidade parecer razoável a olho.

Numa segunda iteração, os limiares foram cruzados com a obra *Measurement of Joint Motion: A Guide to Goniometry*, de Norkin e White [Norkin e White, 2016], que documenta a amplitude de movimento articular e, nalguns capítulos, a amplitude necessária para atividades funcionais concretas. Um cuidado importante ao usar esta fonte é a convenção de ângulo: esta obra mede a flexão como o desvio a partir da posição anatómica neutra (perna esticada corresponde a 0°), enquanto o sistema aqui desenvolvido mede o ângulo interior entre os três pontos de referência (perna esticada corresponde a 180°). A conversão entre as duas é direta, $\theta_{\text{interior}} = 180^\circ - \theta_{\text{flexão}}$.

No Agachamento, Rowe e colaboradores, citados em [Norkin e White, 2016] (p.332), definem que é necessária uma flexão mínima de 110° do joelho para atividades funcionais básicas (andar, subir escadas, levantar de uma cadeira) e que uma flexão de apenas 90° "não é suficiente para atividades normais". Convertidos para a convenção do sistema, estes dois valores correspondem exatamente aos 70° (limiar ideal) e 90° (limiar de contagem) já usados na primeira iteração, confirmando-os sem necessidade de ajuste.

Na Flexão de Braços, o limiar ideal (70° , correspondendo a 110° de flexão do cotovelo) fica dentro da amplitude funcional documentada pela mesma obra (flexão máxima funcional de aproximadamente 140°), pelo que se mantém deliberadamente mais rigoroso sem ser anatomicamente irrealista. O limiar de contagem (90° , 20° acima do ideal) não tem fonte independente.

No Lunge, para o qual esta obra não apresenta valor de referência, o limiar ideal passou por várias iterações: começou nos 80° , foi ajustado para 87° depois de observação visual mostrar que rejeitava repetições bem executadas, e fixou-se nos 90° depois de o joelho monitorizado ter sido trocado do traseiro para o da frente (Secção 3.3), o que exigiu recalibrar também o limiar de contagem de 107° para 115° . Nenhum destes valores tem fonte independente na literatura.

Para a Prancha, também sem valor de referência disponível, manteve-se o valor já usado pela janela de detecção original (170°), sem introduzir um novo alvo.

3.3 Abordagem de Avaliação de Forma

O ângulo θ usado em todos os exercícios é calculado a partir de três landmarks, A , B e C , onde B é sempre o vértice (a articulação central). Usam-se apenas as coordenadas x e y de cada ponto no plano da imagem; a coordenada de profundidade z , também estimada pelo MediaPipe, não é utilizada. O ângulo obtém-se a partir do produto interno dos vetores $\vec{BA}$ e $\vec{BC}$:

$$\theta = \arccos \left(\frac{\vec{BA} \cdot \vec{BC}}{|\vec{BA}| |\vec{BC}|} \right)$$

Esta é a fórmula geométrica aplicada a todos os ângulos definidos na Secção 3.2.

A lógica de detecção assenta numa máquina de estados finita de três estados, **AT_TOP**, **DESCENDING** e **ASCENDING**, que processa este ângulo a cada frame, ilustrada na Figura 3.2. A partir de **AT_TOP**, o sistema entra em **DESCENDING** assim que o ângulo desce abaixo do início da fase excêntrica, definido como o limiar de extensão menos uma tolerância de 15° (no Agachamento, por exemplo, $160^\circ - 15^\circ = 145^\circ$), de forma a só iniciar a fase excêntrica quando o movimento de descida é inequívoco. Em **DESCENDING** há duas transições possíveis: se o ângulo atingir o limiar de contagem, a fase excêntrica termina e o sistema avança para **ASCENDING**; se o ângulo regressar ao topo sem atingir esse limiar, a tentativa é descartada e o sistema volta a **AT_TOP** sem contar repetição. Em **ASCENDING**, ao atingir o limiar de extensão, a repetição é contabilizada.

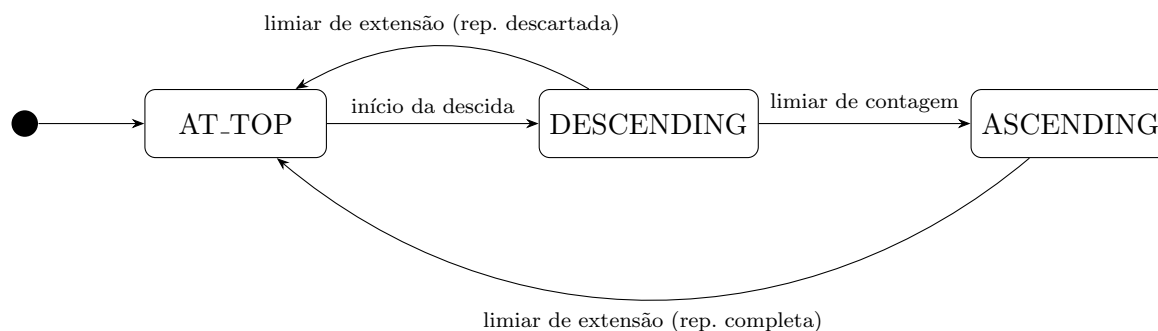


Figura 3.2: Máquina de estados da contagem de repetições.

A qualidade de cada repetição parte de 100 pontos, dos quais se subtraem penalizações independentes, não uma distribuição percentual que soma 100. No pior caso, com falha máxima nas três categorias em simultâneo, uma repetição contabilizada fica com 15 pontos, nunca 0: o score reflete sempre que a repetição foi de facto executada até ao limiar de contagem, mesmo que com qualidade fraca.

- **Dedução de ROM (até 40 pontos):** a repetição só é contabilizada a partir do limiar de contagem, mais permissivo do que o limiar ideal (valores completos por exercício na Secção 3.2). Entre os dois limiares, a penalização cresce de forma linear e contínua até ao máximo de 40 pontos;
- **Cadência Excêntrica (até 30 pontos):** A duração ideal da fase excêntrica varia por exercício: entre 2.0 e 4.0 segundos no Agachamento, e entre 1.5 e 3.0 segundos na Flexão de Braços e no Lunge. Descer demasiado depressa deduz 30 pontos; descer demasiado devagar deduz 10 pontos;
- **Cadência Concêntrica (até 15 pontos):** A subida deve durar entre 0.5 e 2.0 segundos, igual em todos os exercícios. Subir demasiado depressa deduz 15 pontos; subir demasiado devagar deduz 10 pontos.

A cadência excêntrica pesa o dobro da concêntrica (30 contra 15 pontos) porque o controlo da fase excêntrica tem uma influência documentada maior sobre a qualidade da execução: durante a descida, o músculo trabalha sob alongamento em vez de encurtamento, e perder o controlo da velocidade transfere parte da carga para estruturas passivas (tendões, ligamentos) em vez de esta ser absorvida ativamente pelo músculo. Wilk, Zajac e Tufano [Wilk et al., 2021], numa revisão sobre o efeito da cadência de movimento no treino de força, documentam que a duração da fase excêntrica tem uma influência mais consistente sobre os ganhos de força e hipertrofia do que a duração da fase concêntrica, o que motiva a maior penalização por perda de controlo nessa fase.

O mecanismo descrito acima, a máquina de estados e as três categorias de dedução, aplica-se aos três exercícios contados por repetição (Agachamento, Flexão de Braços e Lunge). A Prancha Isométrica, por não ter fases concêntrica e excêntrica distintas, é

avaliada de forma diferente: em vez de percorrer os três estados, o sistema verifica continuamente se o ângulo do tronco se mantém dentro da janela de detecção (155° a 185° , Secção 3.2); enquanto isso se mantiver verdadeiro, o tempo de prancha acumula-se e o score resulta do desvio médio do ângulo em relação ao valor ideal (170°) ao longo de todo o segmento contínuo, e não de penalizações separadas por fase.

O Lunge tem ainda uma particularidade que a máquina de estados por si só não capta: como o joelho monitorizado é o da perna da frente, e o utilizador troca de perna dianteira a cada repetição, o sistema identifica a cada frame qual a perna atualmente à frente e mede o ângulo dessa perna. Um ciclo completo de **DESCENDING** para **ASCENDING** é sempre contabilizado como repetição, mesmo que o utilizador não tenha alternado a perna desde a repetição anterior; nesse caso, o sistema emite um aviso a incentivar a alternância em vez de recusar a contagem, para não perder repetições válidas por falhas pontuais de detecção da perna dianteira.

3.3.1 Exemplo Prático de uma Repetição

Para tornar concreta a interação entre a máquina de estados e a fórmula de pontuação, considere-se o Agachamento (limiar ideal 70° , limiar de contagem 90° , cadência excêntrica ideal entre 2.0 e 4.0s, concêntrica ideal entre 0.5 e 2.0s), acompanhando duas repetições distintas do mesmo utilizador.

Na Repetição 1, o utilizador parte de **AT_TOP** com o joelho a 165° . Ao descer abaixo dos 145° (início da fase excêntrica), o sistema transita para **DESCENDING** e começa a contar o tempo. O ângulo mínimo atingido é 65° , ultrapassando o limiar de contagem (90°), pelo que o sistema transita para **ASCENDING**; a fase excêntrica durou 2.5 segundos. Na subida, o joelho volta a ultrapassar os 160° ao fim de 1.2 segundos (fase concêntrica), o que completa a repetição e a devolve a **AT_TOP**. Como o ângulo mínimo (65°) ultrapassa o limiar ideal (70°), a dedução de ROM é nula; as durações de 2.5s e 1.2s ficam dentro dos intervalos ideais das respetivas fases, pelo que também não há dedução de cadência. O score final é 100.

Na Repetição 2, o mesmo percurso repete-se, mas o utilizador só desce até 82° antes de inverter o movimento. Como 82° ainda ultrapassa o limiar de contagem (90°), a repetição continua a ser contabilizada, mas já não atinge o limiar ideal (70°), pelo que a dedução de ROM deixa de ser nula: penalização = $\frac{82-70}{90-70} \times 40 = 24$ pontos. Mantendo as mesmas cadências da Repetição 1 (sem dedução adicional), o score final é $100 - 24 = 76$.

A Tabela 3.1 resume as duas repetições.

Repetição	Ângulo mínimo	Cadências (exc. / conc.)	Score final
1 (profunda)	65°	2.5s / 1.2s (dentro do ideal)	100
2 (rasa)	82°	2.5s / 1.2s (dentro do ideal)	76

Tabela 3.1: Exemplo prático de duas repetições de Agachamento e a pontuação resultante

Capítulo 4

Implementação do Modelo

Neste capítulo, descrevemos a arquitetura e detalhes técnicos de concretização da aplicação nativa desenvolvida para Android utilizando a linguagem Kotlin e o ecossistema Jetpack Compose.

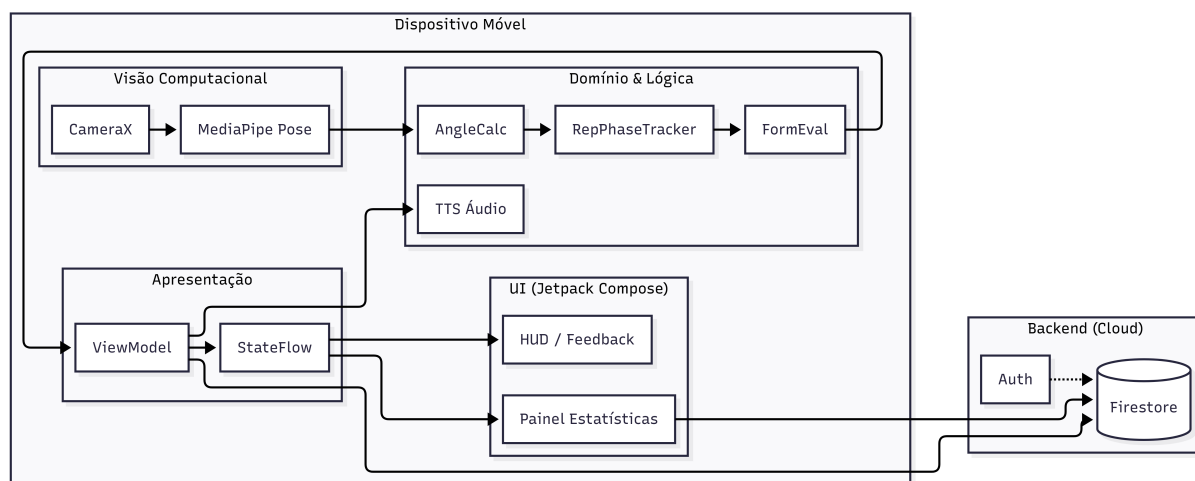


Figura 4.1: Diagrama da arquitetura do projeto

4.1 Arquitetura e Fluxo de Visão por Computador

Antes de se chegar a esta arquitetura, foi construído um protótipo em Python (OpenCV e MediaPipe), a correr num computador, para validar a viabilidade da deteção de pose e da lógica de contagem de repetições. Confirmada a abordagem, mas reconhecendo que manter o motor de deteção como um sistema à parte do produto final traria complexidade de integração sem benefício real, decidiu-se reescrever a lógica de produção nativamente em Kotlin, diretamente integrada na app Android; o protótipo permanece no repositório (03_Implementacao/POC_Python) como prova de conceito histórica.

A aplicação segue a arquitetura **MVVM (Model-View-ViewModel)**, que separa a lógica de negócio (avaliação de forma, máquina de estados) da camada de interface, construída em Jetpack Compose. O Compose foi adotado para toda a interface por

reduzir o código repetitivo face ao sistema tradicional de *Views* em XML e por permitir desenhar os gráficos de progresso do painel analítico (Secção 5.3) diretamente em Canvas, sem depender de bibliotecas de gráficos de terceiros. A captura de imagem recorre à biblioteca CameraX em vez da Camera2 API de baixo nível, por abstrair variações de hardware entre fabricantes e reduzir a complexidade de gestão do ciclo de vida da câmara. O pipeline de processamento de imagem da câmara assenta na seguinte estrutura:

1. **Captura de Imagem (CameraX):** Configura a câmara frontal do dispositivo móvel para obter frames no formato RGBA_8888 de forma contínua e assíncrona;
2. **Pré-processamento:** Roda o bitmap da frame de acordo com a rotação física do dispositivo e aplica um efeito de espelho (*mirror*) para que a experiência do utilizador seja intuitiva;
3. **Scheduler do MediaPipe:** O motor BlazePose opera em modo *Live Stream*. A estratégia de descarte de frames do CameraX (detalhada adiante) garante que a cadência de inferência não fica presa a frames antigas;
4. **Lógica de Negócio Local:** Após a inferência, a lista de 33 coordenadas normalized 3D é canalizada para a classe Kotlin correspondente ao exercício ativo.

O MediaPipe Pose Landmarker segue a arquitetura leve de duas fases descrita por Bazarevsky et al. [Bazarevsky et al., 2020]: um detetor de corpo inteiro localiza a região de interesse do utilizador apenas na primeira frame (ou sempre que o rastreio é perdido), e um modelo de rastreio estima os 33 landmarks a partir dessa região em cada frame seguinte. Isto evita repetir a deteção completa do corpo em todas as frames, reduzindo o custo computacional.

Este projeto utiliza especificamente a variante `pose_landmarker_lite`, a mais leve das três disponibilizadas pela Google (Lite, Full, Heavy), que os próprios autores do BlazePose reportam atingir até 31 frames por segundo num Google Pixel 2 [Bazarevsky et al., 2020], favorecendo a fluidez em tempo real face à precisão adicional das variantes mais pesadas. Não é imposta nenhuma resolução ou frame rate fixos: o `ImageAnalysis` do CameraX é configurado com a estratégia `STRATEGY_KEEP_ONLY_LATEST`, que descarta frames antigas em fila e analisa sempre a mais recente, deixando a cadência efetiva de inferência adaptar-se automaticamente à capacidade de cada dispositivo, com as frames entregues ao MediaPipe no formato RGBA_8888 através da câmara frontal (`CameraSelector.DEFAULT_FRONT_CAMERA`).

4.2 Módulos de Código Principais

O processamento cinemático e as interações de treino apoiam-se num conjunto de classes Kotlin que concretizam diretamente o modelo apresentado no Capítulo 3:

- **AngleCalculator:** Implementa a fórmula geométrica $\theta = \arccos \left(\frac{\vec{BA} \cdot \vec{BC}}{(|\vec{BA}| |\vec{BC}|)} \right)$ apresentada na Secção 3.3, calculada a partir das coordenadas x, y de três landmarks;

- **ExerciseConfig:** Estrutura de dados que armazena, por exercício, os limiares definidos na Secção 3.2 (limiar ideal, limiar de contagem, limiar de extensão) e os intervalos de cadência ideal usados na Secção 3.3;
- **RepPhaseTracker:** Implementa a máquina de estados `AT_TOP/DESCENDING/ASCENDING` da Figura 3.2, medindo a duração em milissegundos das fases descendente (excêntrica) e ascendente (concêntrica) e registrando a amplitude de movimento mínima da articulação central durante a repetição;
- **FormEvaluator:** Implementa as penalizações de score descritas na Secção 3.3 e devolve strings para a correção física;
- **TTSHelper:** Wrapper do motor de síntese de voz nativo do Android. Implementa um limite temporal de repetição (cooldown) para evitar sobreposição de fala, com exceção da conclusão de repetições e números de contagens, que são pronunciados de imediato;
- **OverlayView:** Vista customizada que desenha o esqueleto corporal do utilizador e elementos informativos (HUD), incluindo avisos de calibração.

Cada um dos quatro exercícios (Agachamento, Flexão de Braços, Lunge e Prancha) é implementado numa classe própria (`SquatExercise`, `PushUpExercise`, `LungeExercise` e `PlankExercise`, respetivamente), todas a implementar uma interface comum `Exercise`. Os três primeiros combinam `RepPhaseTracker` e `FormEvaluator` exatamente como descrito acima; a Prancha, por avaliar uma postura estática e não uma repetição (Secção 3.3), não usa nenhuma das duas classes, implementando antes a sua própria lógica de acumulação de tempo sob tensão e desvio médio do ângulo. Cada repetição concluída pelos três primeiros exercícios (ou cada segmento de prancha concluído) gera uma instância de `RepMetrics`, guardada no histórico do exercício (`repHistory`). A Figura 4.2 resume as relações entre estas classes.

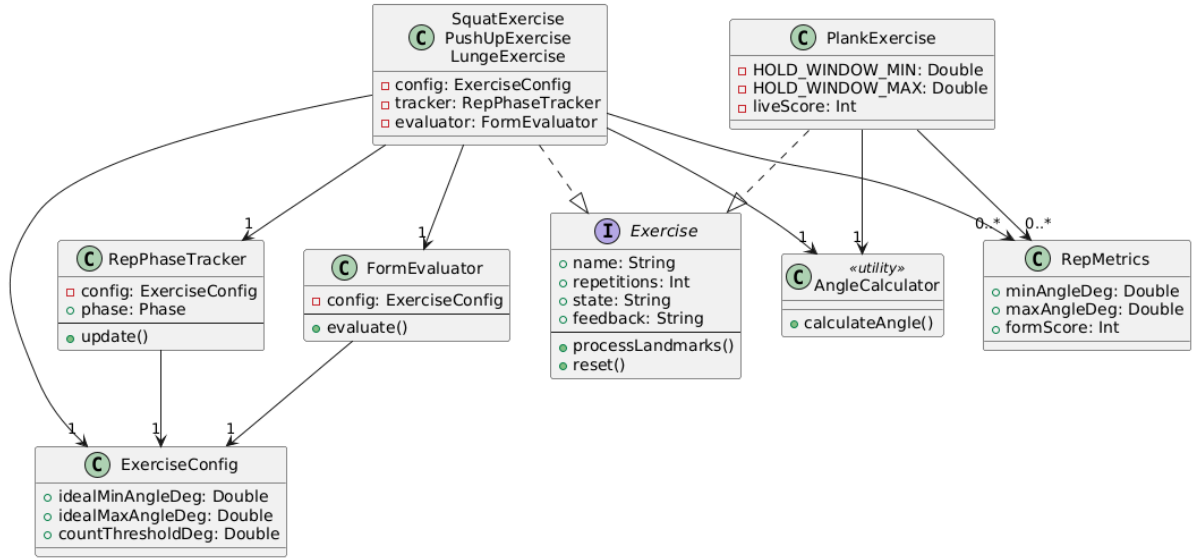


Figura 4.2: Diagrama de classes UML do motor de avaliação de exercícios

A orquestração do treino, por sua vez, é feita por duas *Activities* distintas conforme o tipo de sessão: o *MainActivity* conduz planos multi-exercício definidos pelo utilizador através de um *WorkoutManager*, que percorre sequencialmente uma lista de *TrainingStep* (cada um associando um *Exercise* a uma meta de repetições ou de tempo, incluindo pausas via *RestExercise*); já o *ExerciseTestActivity* conduz uma sessão guiada de exercício único, com deteção de posição inicial (*StartPositionDetector*) e contagem decrescente antes de começar a contar repetições. Ambas partilham o *OverlayView* (esqueleto e HUD sobre a câmara) e o *TTSHelper* (indicações por voz). A Figura 4.3 resume estas relações.

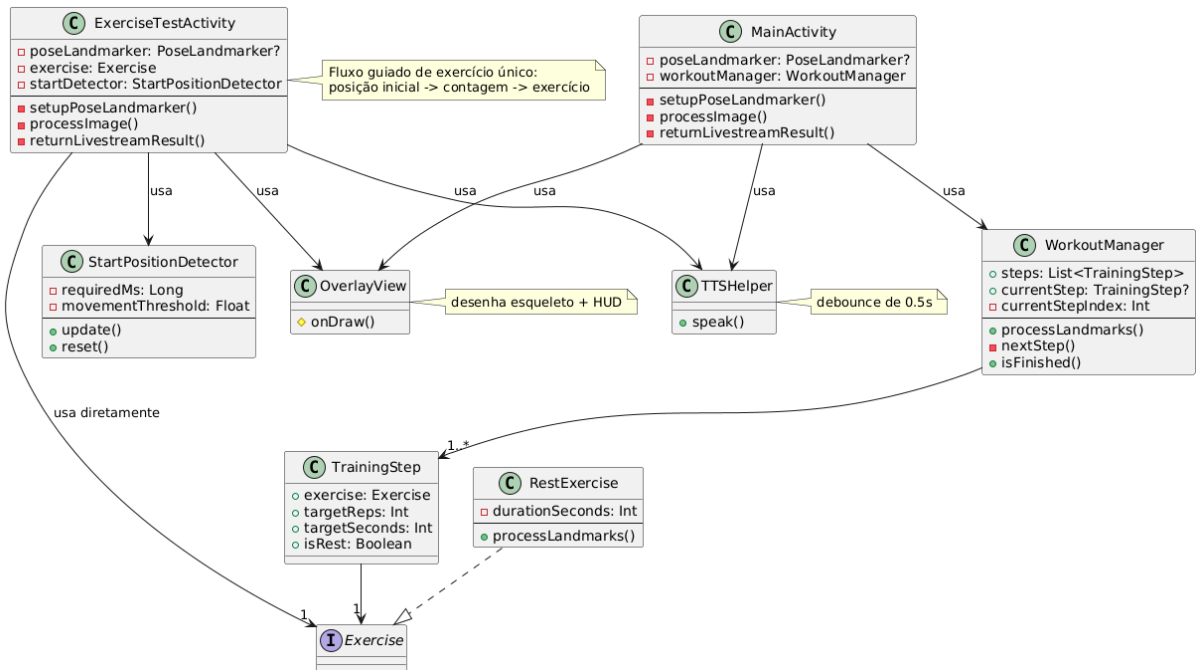


Figura 4.3: Diagrama de classes UML da orquestração do treino.

O fluxo de navegação entre os ecrãs da aplicação (do *login* ao resultado final de um treino) é apresentado no Apêndice A, Figura A.1.

4.3 Integração de Base de Dados e Gamificação

Quando o utilizador conclui um treino, a aplicação calcula as calorias queimadas e regista os dados no Cloud Firestore:

- Grava os dados do treino na coleção `/users/{userId}/workouts`;
- Executa uma **Transação do Firestore** para atualizar de forma segura os campos `xpPoints` e `level` no perfil principal do utilizador. Desta forma, previne-se inconsistência de dados em escritas concorrentes.

A arquitetura NoSQL da aplicação distribui os dados numa hierarquia onde o documento do utilizador (`/users/{userId}`) guarda todos os agregados (Total de Calorias, Pontuação de Cadência Semanal, Pontos de Experiência), enquanto o registo de cada sessão fica numa subcoleção (`/users/{userId}/workouts`).

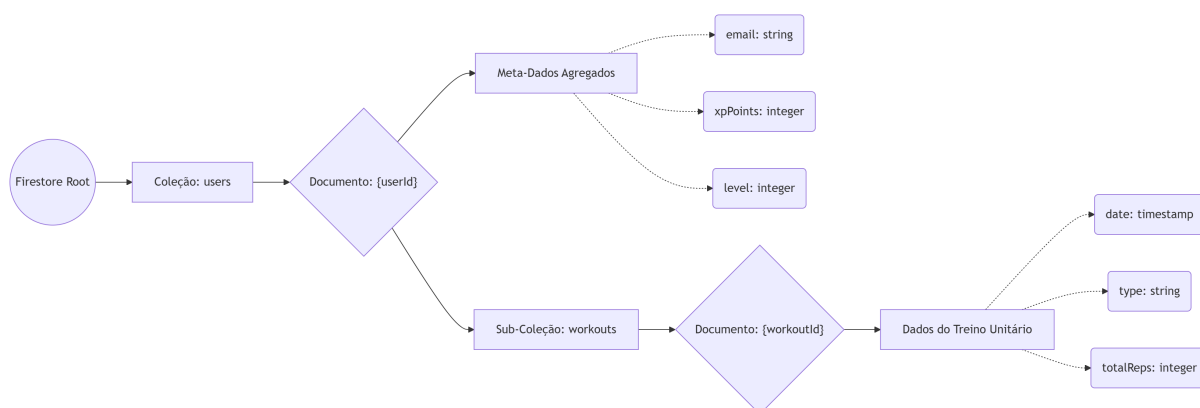


Figura 4.4: Diagrama da estrutura e esquema de dados.

A decisão de executar a atualização dos campos agregados (`xpPoints`, `level`) em tempo de escrita, técnica conhecida como *Write-Time Aggregation*, é uma boa prática em sistemas NoSQL escaláveis. Num protótipo com um reduzido número de utilizadores, a diferença de performance é impercetível; contudo, se a aplicação tiver de calcular a pontuação total do utilizador a partir de milhares de documentos iterados na coleção `workouts` a cada leitura, as *queries* do Firebase tornar-se-ão progressivamente mais lentas e mais dispendiosas à medida que a coleção cresce (complexidade $O(N)$, linear no número de documentos lidos). Ao agregar os dados nativamente no momento em que a sessão é gravada, garantimos que qualquer funcionalidade concorrente (como as *Leaderboards* globais do sistema de gamificação) acede a resultados definitivos numa leitura atómica $O(1)$.

Capítulo 5

Validação e Testes

A validação da aplicação foi realizada recorrendo a testes automatizados de compilação, simulações de base de dados e validações experimentais com utilizadores.

5.1 Validação e Compilação Automatizada

A integridade do código fonte foi verificada através de *builds* contínuos executados com a ferramenta Gradle:

- **Compilação de Kotlin:** Executou-se a tarefa `compileDebugKotlin` para validar tipos estáticos e resolver dependências de bibliotecas de terceiros;
- **Geração de APK:** A tarefa `assembleDebug` empacotou com sucesso o ficheiro APK assinado para instalação e depuração física.

5.2 Testes no Dispositivo e Limitações Físicas

Uma limitação identificada nas ferramentas nativas do MediaPipe (PoseLandmarker) é o facto de compilarem binários nativos de partilha de código (`.so`) maioritariamente para a arquitetura ARM. Ao testar a aplicação em emuladores x86_64 em sistemas Windows, a inicialização do pose landmarker falhava. O código passou a incluir um bloco `try-catch` para desativar a câmara com segurança e mostrar um aviso ao programador/utilizador para correr a aplicação exclusivamente num dispositivo físico ARM.

Em testes reais num smartphone físico moderno (Android 13, CPU Octa-core ARM), o processador local manteve uma taxa de frames de inferência estável próxima dos 25-30 FPS com tempo de computação de regras cinemáticas inferior a 1ms por frame, validando o modelo Edge Computing adotado.

5.3 Validação de Analítica e Estado Social

Os testes de painel interativo e rede social cobriram:

- **Seeding de Dados:** Testou-se a lógica de auto-seeding inserindo 5 treinos passados simulados. O painel estatístico em Jetpack Compose Canvas interpretou corretamente as datas e desenhou barras com altura proporcional às repetições completadas;
- **Limpeza de Histórico:** O botão "Clear History" eliminou de forma limpa todas as entradas na sub-coleção remota, atualizando instantaneamente os gráficos para o estado vazio;
- **Rede Social:** Validou-se a mecânica de aceitação de pedidos de amizade entre dois perfis distintos na consola Firebase, confirmando que a transação atômica adiciona os códigos correspondentes em simultâneo a ambos os utilizadores.

5.4 Testes de Usabilidade e Validação Prática

A validação da interface e da precisão do sistema em ambiente real foi conduzida através de testes de usabilidade estruturados. O protocolo envolveu 7 participantes, segmentados em três grupos com diferentes níveis de literacia tecnológica:

- **Grupo A (Literacia Baixa - 2 participantes):** Foco na avaliação da autonomia inicial, compreensão do vocabulário da interface e dependência de apoio visual.
- **Grupo B (Literacia Média - 3 participantes):** Foco no teste de fluxos de treino padrão e calibração da câmara em ambientes domésticos restritos.
- **Grupo C (Literacia Avançada - 2 participantes):** Foco em testar os limites do algoritmo a ritmos elevados de execução e com utilizadores de estatura elevada (superiores a 1,90m).

A avaliação quantitativa recorreu à escala System Usability Scale (SUS), através de um questionário digital. Embora a amostra de utilizadores ($N = 7$) seja estatisticamente reduzida, a intenção primordial desta avaliação não foi validar o sistema para lançamento em massa, mas sim obter um grupo de controlo preliminar que orientasse as decisões de desenvolvimento adotadas até à data. O *score* global da aplicação obteve uma média de **66.42** pontos num máximo de 100 ($M = 66.42$, $SD = 19.62$), aproximando-se da média dita padrão da indústria (68 pontos), e expõe, em simultâneo, lacunas concretas no fluxo de utilização.

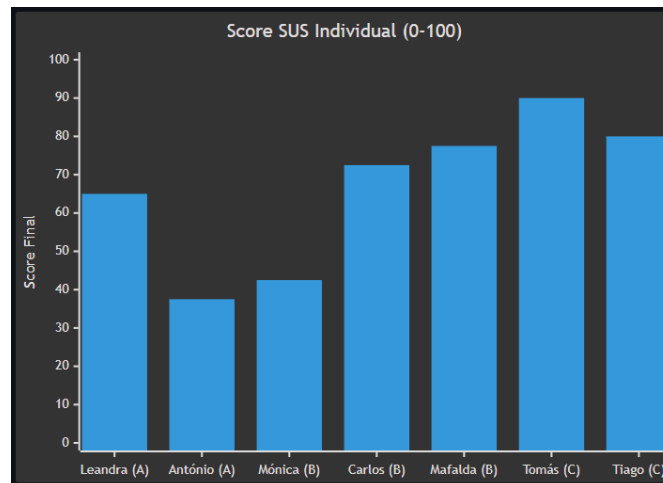


Figura 5.1: Distribuição dos resultados SUS por literacia tecnológica.

Contudo, a análise qualitativa transversal (observação direta das métricas de erro e entrevistas pós-treino) permitiu identificar pontos críticos que resultaram numa correção ágil da aplicação, concretizando as seguintes otimizações:

- **Interface e Idioma:** A ausência de tradução (aplicação nativamente em inglês) gerou entropia e hesitação no ecrã de registo inicial, especialmente nos elementos mais envelhecidos do Grupo A. Em resposta direta, implementou-se um seletor de linguagem (*Language Toggle*) suportado por `AppCompatDelegate`, localizando a interface em tempo real para *pt-PT*.
- **Pacing do Feedback Sonoro (TTS):** Em ritmos elevados, o motor Text-to-Speech nativo do Android gerou sobreposição acústica (frases emitidas em simultâneo). Identificou-se a necessidade de um silêncio (debounce) forçado de 0.5 segundos entre instruções de voz, agora codificado na classe `TTSHelper`, para garantir a clareza do feedback sonoro.
- **Visibilidade Funcional à Distância:** À distância necessária para o enquadramento completo do esqueleto corporal do atleta (entre 2.5m a 4.1m), o texto do ecrã (HUD) tornou-se ilegível, induzindo quebras da postura lombar nos utilizadores que tentavam focar os olhos no visor. Por conseguinte, foi aplicada uma escala de redimensionamento às fontes de contexto e contagem (aumentadas até 150 pontos).
- **Fidelidade Cinemática por Oclusão (Lunges):** Enquanto os exercícios de perfil (Agachamentos e Flexões de Braços) apresentaram taxas de falha residuais, os Afundos (*Lunges*), avaliados de frente para a câmara, apresentaram elevadas taxas de falha. Este falso negativo decorreu da oclusão visual da perna traseira pela perna da frente, mais próxima da câmara, o que corrompia os ângulos calculados a partir dessa perna. A correção não foi um algoritmo de deteção de oclusão, mas uma mudança mais direta: o sistema passou a identificar a cada frame qual a perna está

à frente (pela posição horizontal dos pés) e a monitorizar o joelho dessa perna em vez do traseiro, eliminando o problema na origem em vez de o compensar. Esta mudança obrigou também a recalibrar os limiares de deteção (Secção 3.2.1).

5.5 Produto Final

A consolidação da lógica de negócio com a camada de interface, junto com as correções motivadas pelo feedback dos testes SUS, resultou na versão final da aplicação. A principal otimização desta fase foi a localização integral da interface para português, que resolveu a confusão de navegação reportada pelo grupo de teste.

Ao nível da usabilidade geral, a aplicação segue as convenções de navegação do Android, incluindo um botão de saída da conta (*Sign Out*) na Dashboard. Na preparação da atividade (Figura 5.2), o utilizador beneficia de um modelo flexível: é possível construir planos de treino inteiramente personalizados (desde que o exercício esteja presente no sistema) ou simplesmente iniciar planos pré-construídos.

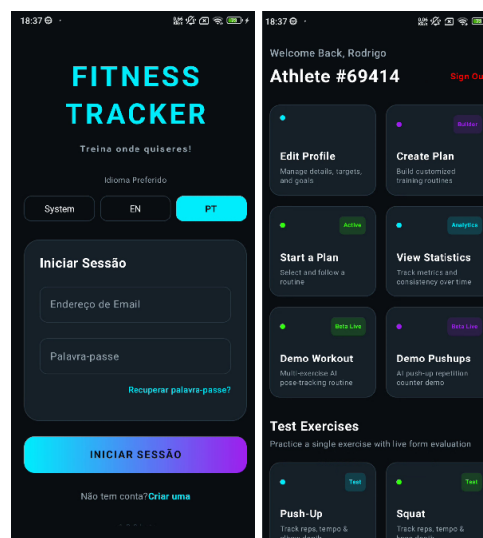


Figura 5.2: Ecrã de login e Menu inicial de seleção de treinos.

A Figura 5.3 ilustra o ciclo de vida completo de uma sessão de treino em ambiente real. Esta sequência demonstra a capacidade do sistema em guiar o utilizador de forma perfeitamente autónoma. Desde a calibração inicial até à transição automática entre exercícios e períodos de descanso, a interface providencia uma experiência contínua, com os apoios visuais e sonoros agora melhor calibrados com pausas curtas para um feedback claro.

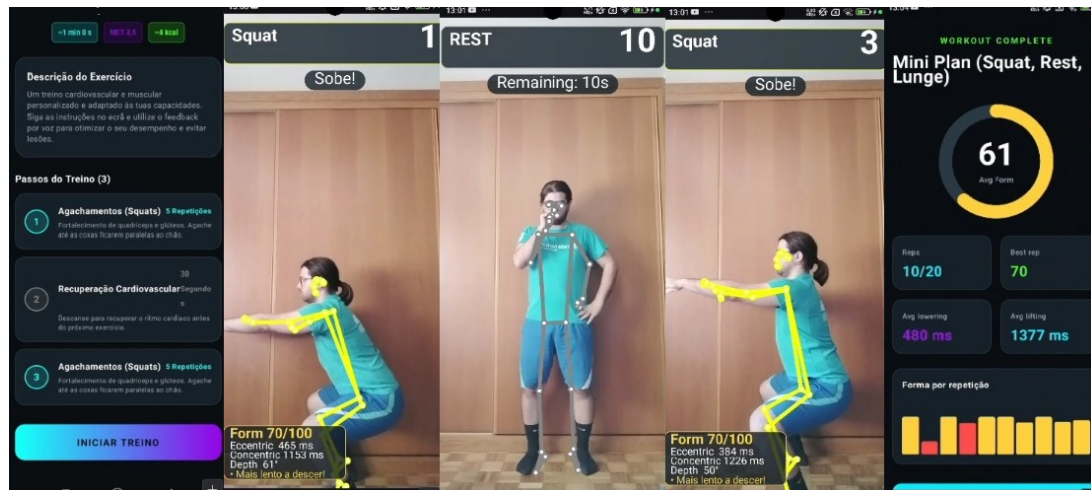


Figura 5.3: Ciclo de utilização da aplicação: Ecrã inicial, Execução do exercício (com HUD), Intervalo de descanso (Rest), Retoma do exercício e Painel de Resultados finais.

Os últimos ecrãs, detalhados na Figura 5.4, materializam o sistema de gamificação, refletindo de imediato uma síntese dos resultados do plano. Este painel prova a viabilidade das decisões de arquitetura de dados (Secção 4.3): as transações de dados pré-computadas na escrita permitem uma leitura imediata dos dados em formato analítico, escalável para uma futura grande dimensão de utilizadores. A métrica apresentada de Pontos de Experiência (XP) atua primariamente como um avaliador da execução técnica do atleta, validando assim a eficácia do *Edge Computing*. Adicionalmente, esta arquitetura deixa as portas abertas e alicerces escaláveis para uma integração futura em lógicas competitivas de *Leaderboard* e interação em rede social, entre várias outras possíveis.

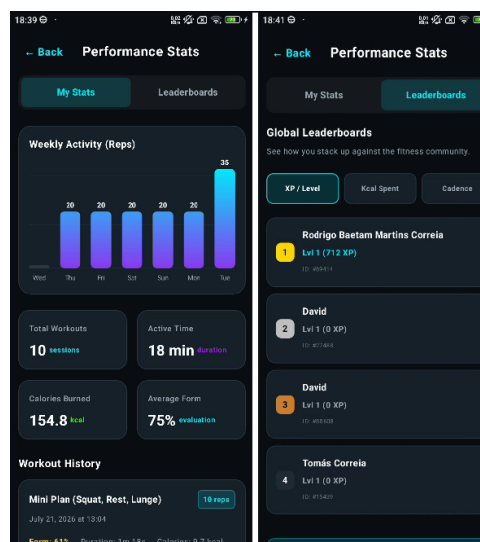


Figura 5.4: Ecrã de visualização de Resultados e Ranking de pontos.

Capítulo 6

Conclusões e Trabalho Futuro

Este trabalho demonstrou a viabilidade de desenvolver um assistente de fitness inteligente num smartphone convencional, sem hardware dedicado. A integração do MediaPipe para estimativa de pose local, aliada a uma máquina de estados em Kotlin e a feedback sonoro (Text-to-Speech), resultou num sistema autónomo capaz de avaliar a amplitude e cadência de exercícios físicos com base na literatura biomecânica.

A validação prática (Secção 5.4) alcançou um *score* SUS de 66.42 e expôs limitações tratadas de forma iterativa, como a sobreposição áudio, as escalas de leitura do HUD e as falhas de deteção no Lunge por oclusão entre pernas.

Como perspectivas de evolução tecnológica, o trabalho futuro estrutura-se em duas grandes frentes: validação de usabilidade e expansão biomecânica. Numa ótica orientada ao mercado e à retenção de utilizadores, planeia-se conduzir testes A/B com novas amostras populacionais para otimizar a representação visual do esforço (HUD) e validar as novas mecânicas de gamificação. Adicionalmente, prevê-se a implementação de desafios competitivos em tempo real entre atletas na rede de amigos. Ao nível da fiabilidade algorítmica, será necessário submeter a nova heurística geométrica do Lunge a testes intensivos de oclusão e estudar soluções preditivas para enquadramentos de câmara não ortodoxos. Por fim, ambiciona-se suportar novos exercícios de membros superiores (como a flexão de bíceps) e expandir o motor de pose para a avaliação postural tridimensional completa (alinhamento ombros-anca-tornozelo), tirando partido da coordenada de profundidade z já extraída nativamente pelo modelo MediaPipe.

Apêndice A

Diagrama de Navegação da Interface

A Figura A.1 apresenta o fluxo de navegação entre os principais ecrãs da aplicação, referido na Secção 4.2. Por clareza, omite-se o `LoaderActivity` (ecrã de transição genérico com *spinner*), usado tanto pelo `LandingActivity` quando já existe sessão iniciada como pelo `DashboardActivity` no acesso direto a treinos (`MainActivity`, `DemoPushUpActivity`, `ExerciseTestActivity`). O `DemoPushUpActivity` é uma demo de duração fixa e atualmente não tem ecrã de resultado.

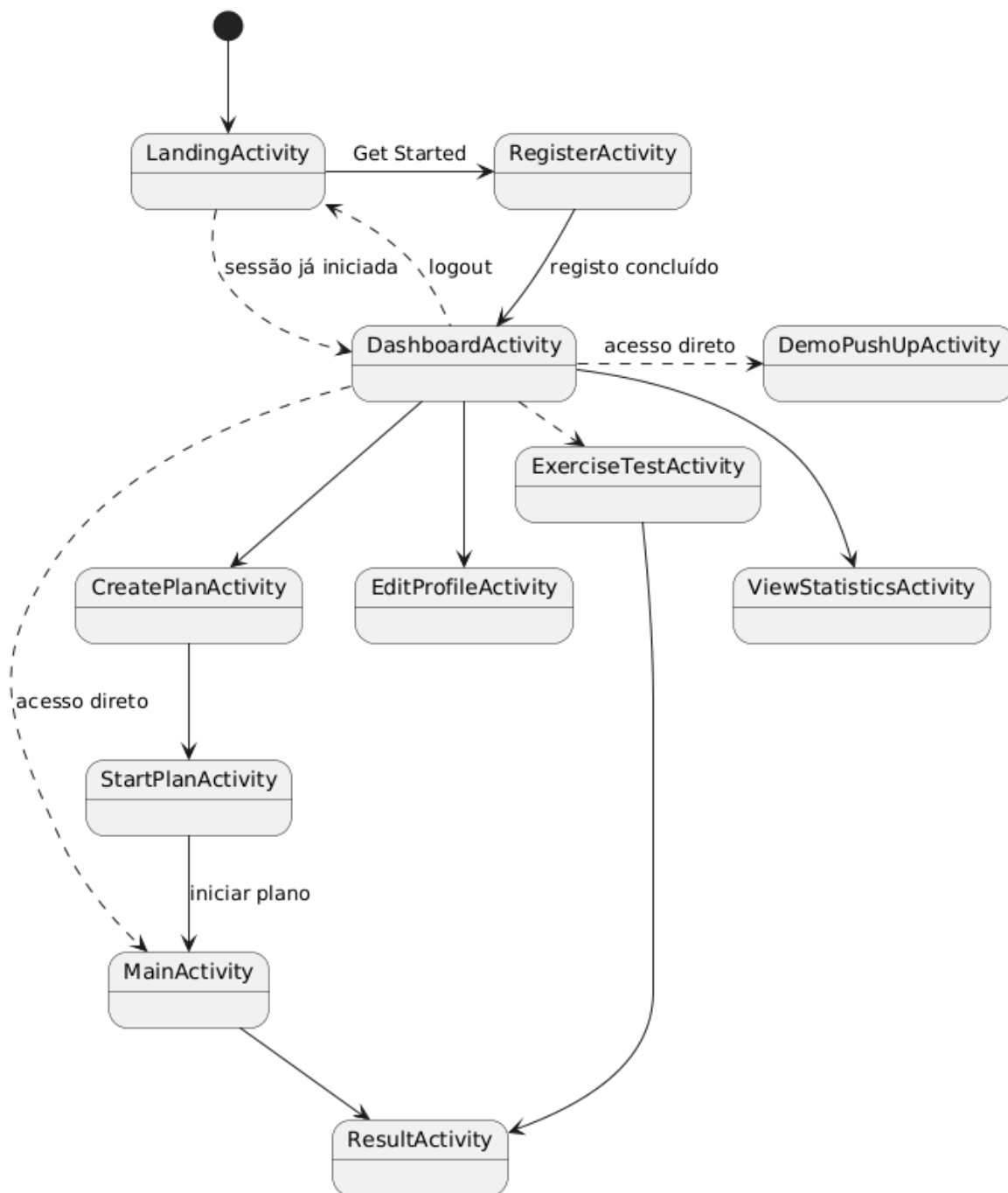


Figura A.1: Fluxo de navegação entre os principais ecrãs da aplicação.

Bibliografia

- [Bazarevsky et al., 2020] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., e Grundmann, M. (2020). BlazePose: On-device real-time body pose tracking. *arXiv preprint arXiv:2006.10204*.
- [Cao et al., 2018] Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., e Sheikh, Y. (2018). OpenPose: Realtime multi-person 2D pose estimation using part affinity fields. *arXiv preprint arXiv:1812.08008*.
- [Garg et al., 2023] Garg, S., Saxena, A., e Gupta, R. (2023). Yoga pose classification: A CNN and MediaPipe inspired deep learning approach for real-world application. *Journal of Ambient Intelligence and Humanized Computing*, 14:16551–16562.
- [Google, 2024] Google (2024). Pose landmark detection guide. https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker. Acedido em 13 de julho de 2026.
- [Maji et al., 2022] Maji, D., Nagori, S., Mathew, M., e Poddar, D. (2022). YOLO-Pose: Enhancing YOLO for multi person pose estimation using object keypoint similarity loss. *arXiv preprint arXiv:2204.06806*.
- [Norkin e White, 2016] Norkin, C. C. e White, D. J. (2016). *Measurement of Joint Motion: A Guide to Goniometry*. F.A. Davis Company, Philadelphia, PA, 5th edition.
- [Wilk et al., 2021] Wilk, M., Zajac, A., e Tufano, J. J. (2021). The influence of movement tempo during resistance training on muscular strength and hypertrophy responses: A review. *Sports Medicine*, 51(8):1629–1650.